

On Celebration:

BCNF decomp

$C \rightarrow D$

$D \rightarrow A$

(:)

Answers.. some people
BECA ??

~~CD~~ ✓

~~DA~~ ✓

$C \rightarrow A$

Indexing!

Index: data structure that lets you look up
something quickly given a search key.

- hash map / hash table

- binary search tree

- B-trees

- B+ trees

- Red/black trees

- array / list (if search key is 0-? integer)

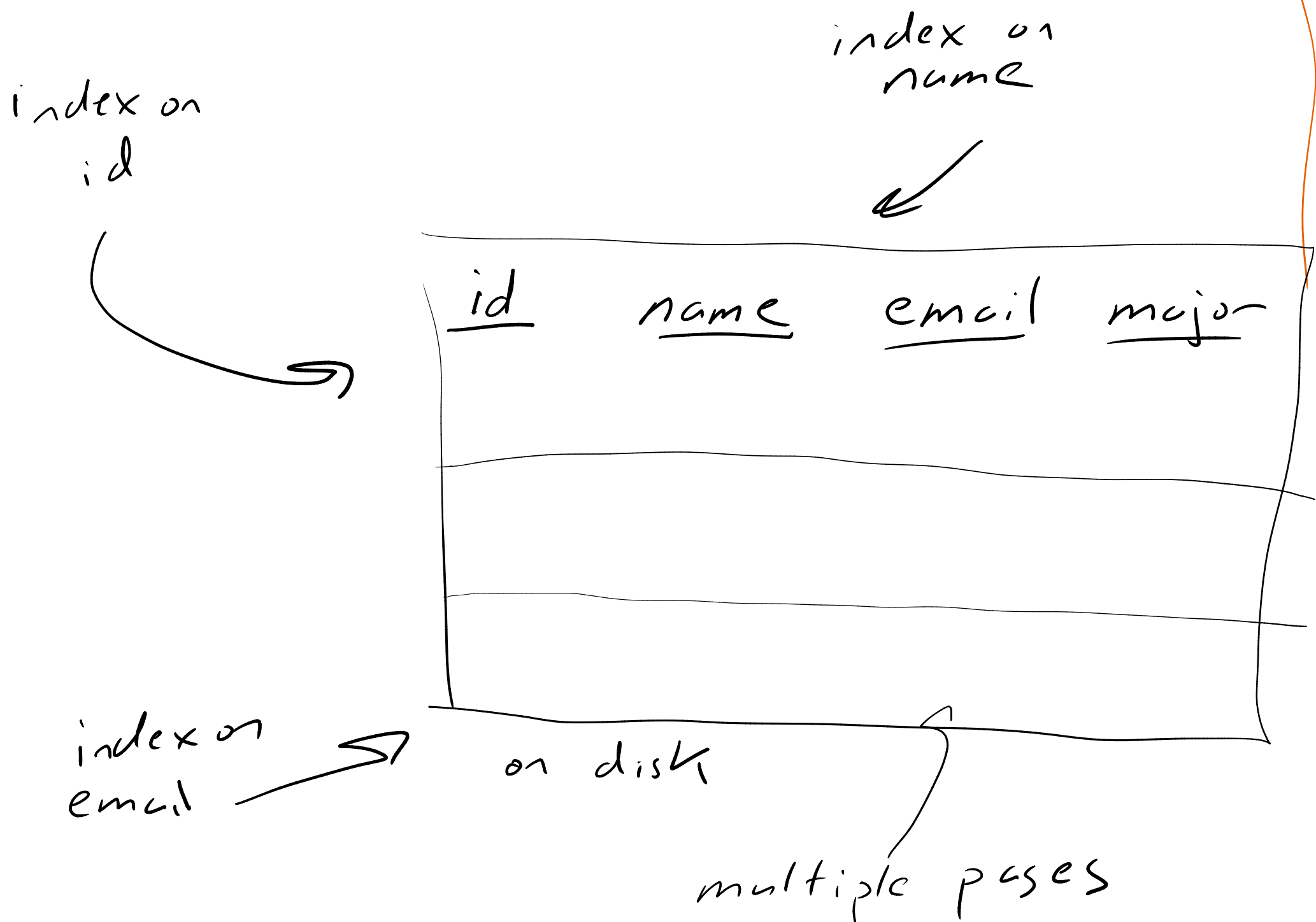
A Python dictionary is implemented via
a hash table

} dictionary
is an ADT
that could
be implemented
w/ any of these

Why again, and what's different?

- In CS 261, assume everything is in memory, and all access time is the same.
- Here
 - memory vs disk
 - seq vs random access
- need to revisit approaches taking this into account

We will often have multiple index on one data source.



Index vocabulary

ordered index - entries in index are arranged based on an ordering of the search key values.

hash map is not an ordered index

↳ B ST is an ordered index

↳ alternative is a hashed index

Primary index - index specifies the ordering of the file (the data itself)

↗ In this student example, perhaps we force it to be sorted on id.

If we do, then any index on id is a primary index.

- also called a clustering index

clustered index

if it's not a primary index, it's a secondary index, or unclustered index

dense index - every search key

appears in the index

eg. index on id - every id is in the index

sparse index - only some search keys

are in there

- Can work, if index is ordered.

Data

<u>index</u>		<u>id</u>
1		1
4	seq search ↓	2
		3
10		4
		6
		9
		10
		11

I want 3

What does an actual DB index look like?

"Simple (not practical) example
"Indexed sequential file"

Primary index - just a list of search keys
(maybe some if space) and a RID
for each.

'record id'
(page id,
slot id)

<u>id</u>	<u>rid</u>
1	(1,0)
3	(1,1)
11	(2,0)
15	(2,1)
18	(3,0)
25	(3,1)

<u>id</u>	<u>name</u>
1	Dave
3	Schiller
11	Olaf
15	Phone
18	Laptop
25	Alison

- unique identification
for finding the
record

← page 1

← page 2

← page 3

Why is this any better than just
searching within the data itself?

"Why not just binary search the data for
the ID you're looking for?"

"Even if you have a good reason, why doesn't that same reason apply to the index?"

"Haven't you just made it worse? Now you have to search the index, and then go to disk to get the data."

Why not binary search data?

Slow to go to storage for each page

→ random lookups

Why doesn't this apply to index?

- Umm... it does, But we can save a little..

Even if index doesn't fit, you get more index entries on a page than you get records.

If records are really large, maybe you only fit a very small number of them on a page.

For this example index, maybe each entry
id, page id, block id 3 * some # of bytes

Huge savings because fewer pages in the index. But storing sequential data is messy - inserts get really expensive.

<u>id</u>	<u>rid</u>
1	(1,0)
3	(1,1)
11	(2,0)
15	(2,1)
18	(3,0)
25	(3,1)

↙ Inserting in here would require sliding everything down, $O(n)$, super slow

—
If you instead store the index as a tree, everything becomes much more efficient

If you use a balanced tree it works even better!

→ B+ tree

- balanced tree

- merge pages properly

* #1 most popular DB index technique.